

Guia de patrones de programacion - Parcial practico

Bridge - Strategy - Builder - Chain of Responsibility - Observer - Flyweight - Repository - Dependency Injection

Objetivo: leer un enunciado, reconocer la estructura del problema y escoger rapidamente el patron adecuado. La guia usa ejemplos sencillos en Java y esta pensada para adaptarlos durante el examen.

1. Mapa rapido para escoger patron

Si el enunciado habla de...	Piensa en...
Varias formas de hacer la misma operacion	Strategy
Dos dimensiones que deben cambiar por separado	Bridge
Construir un objeto complejo paso a paso	Builder
Una solicitud que puede ser atendida por varios objetos en cadena	Chain of Responsibility
Un cambio que debe avisarse automaticamente a varios interesados	Observer
Muchisimos objetos muy parecidos y memoria como problema	Flyweight
Separar la logica del negocio del acceso a datos	Repository
Una clase recibe desde afuera lo que necesita para funcionar	Dependency Injection

Bridge

Que resuelve: Separar una abstraccion de su implementacion para que ambas puedan variar de manera independiente.

Como identificarlo: Busca **dos ejes de cambio**. Ejemplo: tipo de dispositivo + forma de control; reporte + formato; figura + color/renderizador. Si aparecen dos jerarquias que crecerian por separado, Bridge es candidato.

Estructura mental: En vez de heredar todas las combinaciones, una clase *Abstraccion* contiene una referencia a *Implementador*.

Plantilla minima:

```
interface Implementador {
    void operar();
}

class ImplementadorA implements Implementador {
    public void operar() { }
}

class Abstraccion {
    protected Implementador impl;
    public Abstraccion(Implementador impl) {
        this.impl = impl;
    }
    public void ejecutar() {
        impl.operar();
    }
}
```

Palabras/pistas del enunciado: Pistas: "independientemente", "puede cambiar A y B", "varias plataformas y varios tipos", "evitar muchas subclases".

Strategy

Que resuelve: Encapsular varios algoritmos o formas de realizar una tarea y permitir cambiar entre ellos.

Como identificarlo: Busca **una misma tarea con varias estrategias**: pagos, ordenamientos, descuentos, rutas, compresion, autenticacion, etc. Si la decision puede cambiar en tiempo de ejecucion, Strategy encaja muy bien.

Estructura mental: El cliente usa un *Context*, y el Context recibe una interfaz *Strategy*. Cada estrategia implementa el algoritmo.

Plantilla minima:

```
interface Strategy {
    void ejecutar();
}

class EstrategiaA implements Strategy {
    public void ejecutar() { }
}

class Context {
    private Strategy strategy;
    public Context(Strategy strategy) {
        this.strategy = strategy;
    }
    public void ejecutar() {
        strategy.ejecutar();
    }
}
```

Palabras/pistas del enunciado: Pistas: "varios metodos para", "elegir algoritmo", "puede cambiar la forma de calcular/procesar".

Builder

Que resuelve: Construir objetos complejos de forma controlada, normalmente paso a paso, separando construccion de representacion.

Como identificarlo: Busca **muchos atributos opcionales**, configuraciones largas o una construccion que naturalmente ocurre por pasos. Ejemplo: computador con RAM, disco, procesador, GPU.

Estructura mental: Un Builder acumula configuracion y al final devuelve el objeto con *build()*.

Plantilla minima:

```
class Producto {
    String nombre;
    int cantidad;
}

class ProductoBuilder {
    private final Producto p = new Producto();
    public ProductoBuilder nombre(String n) { p.nombre = n; return this; }
    public ProductoBuilder cantidad(int c) { p.cantidad = c; return this; }
    public Producto build() { return p; }
}
```

Palabras/pistas del enunciado: Pistas: "crear paso a paso", "muchas opciones", "configurar antes de construir", "build()".

Chain of Responsibility

Que resuelve: Pasar una solicitud por una cadena de objetos hasta que uno la procese o decida dejarla pasar.

Como identificarlo: Busca **varios posibles receptores** donde no quieres que el emisor conozca cual la atiende. Ejemplo: aprobacion por montos, soporte tecnico nivel 1/2/3, validaciones sucesivas.

Estructura mental: Cada Handler conoce al siguiente. Intenta procesar; si no puede, delega en *next*.

Plantilla minima:

```
abstract class Handler {
    protected Handler next;
    public Handler setNext(Handler next) {
        this.next = next; return next;
    }
    public abstract void manejar(int dato);
}

class HandlerA extends Handler {
    public void manejar(int dato) {
        if (dato < 10) { /* procesa */ }
        else if (next != null) next.manejar(dato);
    }
}
```

Palabras/pistas del enunciado: Pistas: "si no puede, pasa al siguiente", "niveles", "cadena", "el primero que pueda atender".

Observer

Que resuelve: Establecer una relacion uno-a-muchos para notificar automaticamente a varios objetos cuando cambia el estado de otro.

Como identificarlo: Busca **un objeto fuente que cambia y muchos interesados**. Ejemplo: precio de producto que avisa a clientes, sensor que notifica dashboards, canal que avisa a suscriptores.

Estructura mental: Un *Subject* guarda una lista de *Observers*. Cuando cambia, llama a *update()* en todos.

Plantilla minima:

```
interface Observer {
    void update(String mensaje);
}

class Subject {
    private List observers = new ArrayList<>();
    public void addObserver(Observer o) { observers.add(o); }
    public void notifyObservers(String msg) {
        for (Observer o : observers) o.update(msg);
    }
}
```

Palabras/pistas del enunciado: Pistas: "notificar", "suscribirse", "avisar a todos", "cuando cambie X, actualice Y/Z".

Flyweight

Que resuelve: Compartir datos intrinsecos repetidos entre muchisimos objetos para reducir memoria.

Como identificarlo: Busca **gran cantidad de instancias similares**. La memoria importa y parte de la informacion se repite. Ejemplo: 100000 arboles con el mismo tipo/textura; personajes con misma apariencia pero diferente posicion.

Estructura mental: Se separa el estado compartido (*intrinseco*) del estado unico (*extrinseco*). Una *Factory* reutiliza Flyweights.

Plantilla minima:

```
interface Flyweight {
    void mostrar(int x, int y);
}
```

```

}

class FlyweightFactory {
private Map cache = new HashMap<>();
public Flyweight get(String tipo) {
return cache.computeIfAbsent(tipo, k -> new Arbol(k));
}
}

class Context {
int x, y;
Flyweight tipo;
}

```

Palabras/pistas del enunciado: Pistas: "miles/millones de objetos", "ahorrar memoria", "comparten la misma informacion", "solo cambia posicion/id/estado externo".

Repository

Que resuelve: Abstraer el acceso y almacenamiento de entidades para que el dominio no dependa directamente de una lista, archivo o base de datos.

Como identificarlo: Busca **operaciones CRUD** o consultas sobre entidades y una separacion clara entre negocio y persistencia.

Estructura mental: La interfaz Repository define operaciones; una implementacion decide como guardar/buscar. El Service usa el Repository.

Plantilla minima:

```

interface UsuarioRepository {
void guardar(Usuario u);
Usuario buscarPorId(int id);
List obtenerTodos();
void eliminar(int id);
}

class UsuarioRepositoryImpl implements UsuarioRepository {
// lista, archivo o BD
}

class UsuarioService {
private UsuarioRepository repository;
public UsuarioService(UsuarioRepository repository) {
this.repository = repository;
}
}

```

Palabras/pistas del enunciado: Pistas: "guardar, buscar, listar, eliminar", "persistencia", "base de datos", "separar acceso a datos de reglas de negocio".

Dependency Injection

Que resuelve: Proporcionar a una clase sus dependencias desde afuera, en lugar de que ella misma las cree.

Como identificarlo: Busca una clase que **necesita otra clase** y quieres evitar que quede acoplada a una implementacion concreta.

Estructura mental: Lo mas facil para el parcial es **inyeccion por constructor**: la dependencia entra como parametro.

Plantilla minima:

```

interface Notificador {
void enviar(String mensaje);
}

```

```
}

class NotificadorEmail implements Notificador {
public void enviar(String mensaje) { }
}

class Servicio {
private Notificador notificador;
public Servicio(Notificador notificador) {
this.notificador = notificador;
}
}
```

Palabras/pistas del enunciado: Pistas: "recibe la dependencia", "inyectar", "desacoplar", "no crear new Dependencia dentro de la clase".

2. Comparaciones que pueden confundirte

Confusion	Diferencia clave
Strategy vs Bridge	Strategy cambia el algoritmo de una tarea. Bridge separa dos dimensiones que de
Repository vs Dependency Injection	Repository resuelve como acceder a datos . Dependency Injection resuelve como propo
Observer vs Chain of Responsibility	Observer notifica a todos los interesados . Chain pasa una solicitud hasta que alguien la
Builder vs Constructor	Builder es util cuando la construccion es compleja/opcional . Un constructor normal basta p
Flyweight vs Prototype	Flyweight comparte estado para ahorrar memoria. Prototype clona objetos existen

3. Metodo de 20 segundos para el examen

Paso 1. Encuentra el problema que cambia: algoritmo, construccion, notificacion, receptor, memoria, persistencia, dependencia o dos dimensiones.

Paso 2. Busca la señal dominante del enunciado. No intentes usar un patron solo porque "cabe"; busca la estructura que el problema realmente necesita.

Paso 3. Copia tu plantilla y cambia nombres del dominio. Mantén las interfaces y relaciones del patron.

Paso 4. Haz un Main minimo que demuestre el escenario principal.

Paso 5. Si aparecen dos patrones posibles, pregunta: ¿el problema principal es comportamiento intercambiable, desacoplamiento, acceso a datos, notificacion o ahorro de memoria?

4. Recordatorio de bolsillo

Bridge: dos jerarquias que varian por separado. **Strategy:** varios algoritmos. **Builder:** construir paso a paso. **Chain:** pasar al siguiente. **Observer:** notificar suscriptores. **Flyweight:** compartir para ahorrar memoria. **Repository:** encapsular persistencia. **DI:** recibir dependencias desde afuera.